# TaskFlow — Görev ve Proje Yönetim Sistemi REST API

## Backend Geliştirme Proje Envanteri — Kurulum Notu (README)

| | |
|---|---|
| **Eğitim:** | Node.js ile Backend Programlama — Bitirme Projesi |
| **Geliştirici:** | Sefa Kesen |
| **GitHub Deposu:** | https://github.com/sefakesen/taskflow |
| **Teknolojiler:** | Node.js (v18+), Express.js (v4.18), JSON Veri Deposu, Postman |
| **Tarih:** | Ekim 2026 |

---

TaskFlow, bir yazılım şirketinin ekip içerisindeki görevlerini, projelerini ve çalışan sorumluluklarını takip etmesini sağlayan standartlara uygun bir RESTful API backend uygulamasıdır. Bu doküman, projenin kurulum, çalıştırma, dosya mimarisi ve Postman ile doğrulanmış test sonuçlarını içermektedir.

## 1. Sistem Gereksinimleri

- **Node.js:** v18.0.0 veya üzeri (Önerilen: v20 LTS)
- **npm:** v8.0.0 veya üzeri (Node.js ile birlikte gelir)
- **Postman:** REST API uç noktalarının test edilmesi ve doğrulanması için

## 2. Kurulum ve Çalıştırma Adımları

### Adım 1: Projenin Klonlanması

```
git clone https://github.com/sefakesen/taskflow.git
cd taskflow
```

### Adım 2: Bağımlılıkların Yüklenmesi

```
npm install
```

### Adım 3: Sunucunun Başlatılması

```
# Üretim modu (Normal başlatma)
npm start

# Geliştirme modu (Nodemon ile kod değişikliklerinde otomatik yeniden başlatma)
npm run dev
```

Sunucu başlatıldığında varsayılan olarak **http://localhost:3000** portunda dinlemeye başlar. Başarılı açılışta konsolda şu çıktı görüntülenir:

```
TaskFlow API sunucusu baslatildi: http://localhost:3000
```

## 3. Klasör ve Dosya Hiyerarşisi

```
taskflow/
├── docs/
│   ├── screenshots/                    # 5 Zorunlu CRUD Postman test ekran görüntüleri
│   │   ├── 01_gorev_listeleme.png
│   │   ├── 02_gorev_ekleme.png
│   │   ├── 03_gorev_detayi.png
│   │   ├── 04_gorev_guncelleme.png
│   │   └── 05_gorev_silme.png
```

```
│   ├── proje-tasarim-ciktisi.pdf          # Tasarım Çıktısı (Yalnızca PDF Teslimi)
│   ├── proje-tanitim.md                   # Tasarım Çıktısı Metin Dokümantasyonu
│   ├── api-dokumantasyonu.md              # Ayrıntılı API Uç Nokta Kılavuzu
│   ├── test-senaryolari.md                # Postman Test Senaryoları Matrisi
│   └── TaskFlow.postman_collection.json   # Postman v2.1 Test Koleksiyonu
├── logs/
│   └── requests.log                       # Logger middleware kayıt dosyası
├── src/
│   ├── controllers/
│   │   ├── taskController.js              # Görev CRUD ve filtreleme iş mantığı
│   │   └── reportController.js            # Raporlama servisleri iş mantığı
│   ├── data/
│   │   ├── tasks.json                     # JSON dosya tabanlı veri deposu
│   │   └── taskStore.js                   # Veri erişim katmanı (Data Access)
│   ├── middleware/
│   │   ├── errorHandler.js                # Global hata yakalama katmanı
│   │   ├── logger.js                      # İstek loglama middleware'i
│   │   └── validate.js                    # Girdi doğrulama middleware'i
│   ├── routes/
│   │   ├── reportRoutes.js                # Raporlama rotaları
│   │   └── taskRoutes.js                  # Görev rotaları
│   └── app.js                             # Express sunucu giriş noktası
├── .gitignore
├── package-lock.json
├── package.json
└── README.md                              # Kurulum notu
```

## 4. API Endpoint Listesi

Sistem hem `/api/...` standart önekini hem de kök rotaları ( `/...` ) destekler.

### 4.1. Zorunlu CRUD İşlemleri (Aşama 03)

| Metot | Endpoint | Açıklama | Durum Kodu |
|---|---|---|---|
| GET | `/api/tasks` | Tüm görevleri listeler | `200 OK` |
| POST | `/api/tasks` | Yeni bir görev oluşturur | `201 Created` |
| GET | `/api/tasks/:id` | Belirli görevi ID ile sorgular | `200 OK / 404` |
| PUT | `/api/tasks/:id` | Görev bilgilerini günceller | `200 OK / 404` |
| DELETE | `/api/tasks/:id` | Görevi sistemden siler | `200 OK / 404` |

### 4.2. Gelişmiş Özellikler (İsteğe Bağlı Pratik 01)

| Metot | Endpoint | Açıklama |
|---|---|---|
| GET | `/api/tasks?status=completed` | Duruma göre filtreleme (todo, in-progress, completed, cancelled) |
| GET | `/api/tasks?priority=high` | Önceliğe göre filtreleme (low, medium, high, critical) |
| GET | `/api/tasks/search?keyword=backend` | Başlık ve açıklamada anahtar kelime araması |
| GET | `/api/tasks/assignee/:name` | Atanan çalışana göre filtreleme (örn. /assignee/musa) |
| GET | `/api/tasks?page=1&limit=10` | Sayfalama desteği (Pagination) |
| GET | `/api/tasks?sort=createdAt&order=desc` | Alana ve yöne göre sıralama (Sorting) |

### 4.3. Raporlama Servisleri (İsteğe Bağlı Pratik 03)

| Metot | Endpoint | Açıklama |
|---|---|---|
| GET | `/api/reports/summary` | Genel sistem özeti, durum/öncelik dağılımı ve tamamlanma yüzdesi |
| GET | `/api/reports/pending` | Bekleyen görevlerin sayısı ve listesi |
| GET | `/api/reports/completed` | Tamamlanan görevlerin sayısı ve listesi |

## 5. Middleware Mimarisi

- **Logger Middleware (Zorunlu — Aşama 03):** Gelen tüm istekleri HTTP metodu, adresi, durum kodu ve zaman damgasıyla konsola ve `logs/requests.log` dosyasına yazar.
- **Validation Middleware (İsteğe Bağlı Pratik 02):** `POST` ve `PUT` isteklerinde `title` , `priority` , `status` ve `assignee` alanlarını doğrular; geçersiz veride `400 Bad Request` döner.
- **Global Error Handler Middleware:** Beklenmeyen çalışma zamanı hatalarını yakalar ve istemciye `500` durum koduyla standart JSON yanıtı üretir.

## 6. Postman ile Test ve Doğrulama Kılavuzu

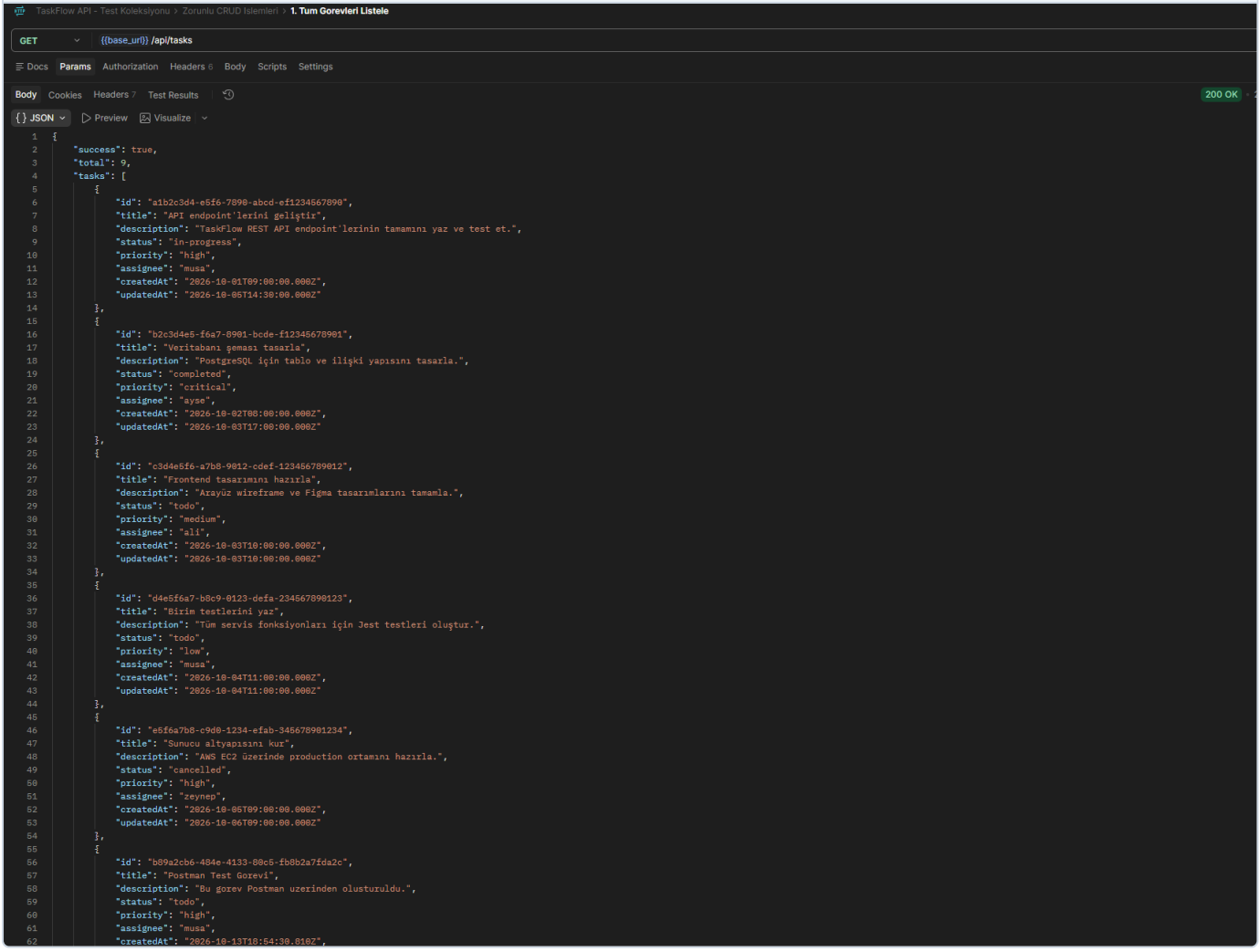
1. Postman uygulamasını açın ve sol üstteki **Import** seçeneğine tıklayın.

2. Repodaki `docs/TaskFlow.postman_collection.json` dosyasını seçin.
3. Koleksiyon değişkeni `base_url` varsayılan olarak `http://localhost:3000` olarak ayarlanmıştır.
4. `POST /api/tasks` isteği çalıştırıldığında dönen `id` otomatik olarak sonraki istekler için `task_id` değişkenine kaydedilir.

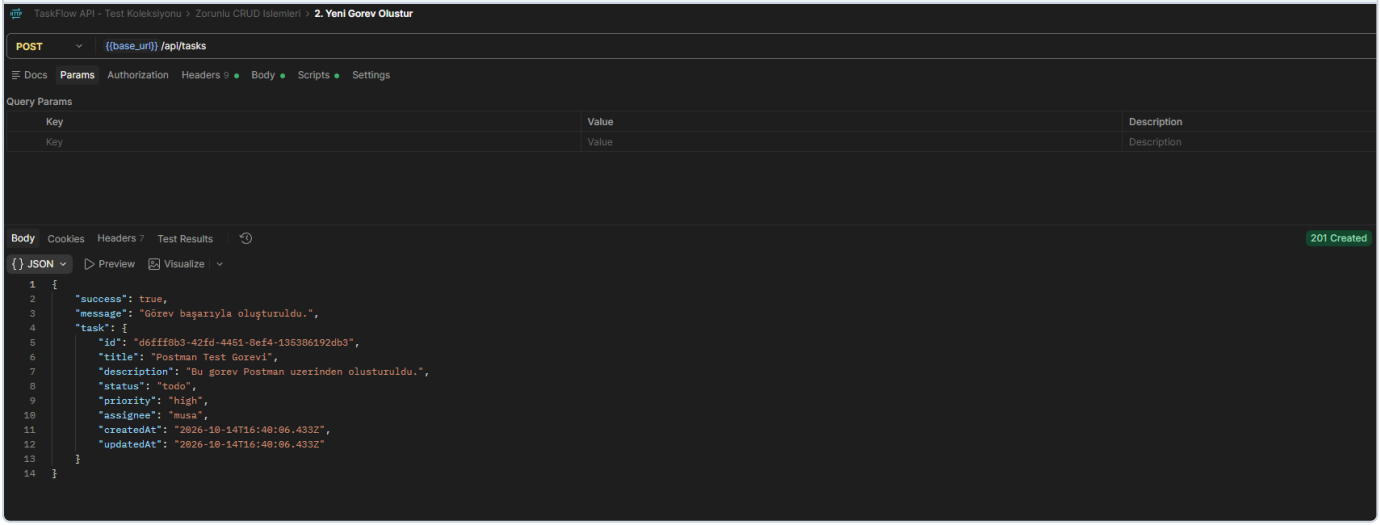
## 7. Test Ekran Görüntüleri ve Doğrulama Çıktıları

Müfredatta belirtilen 5 temel CRUD işleminin başarıyla çalıştığını kanıtlayan Postman ekran görüntüleri aşağıdadır:

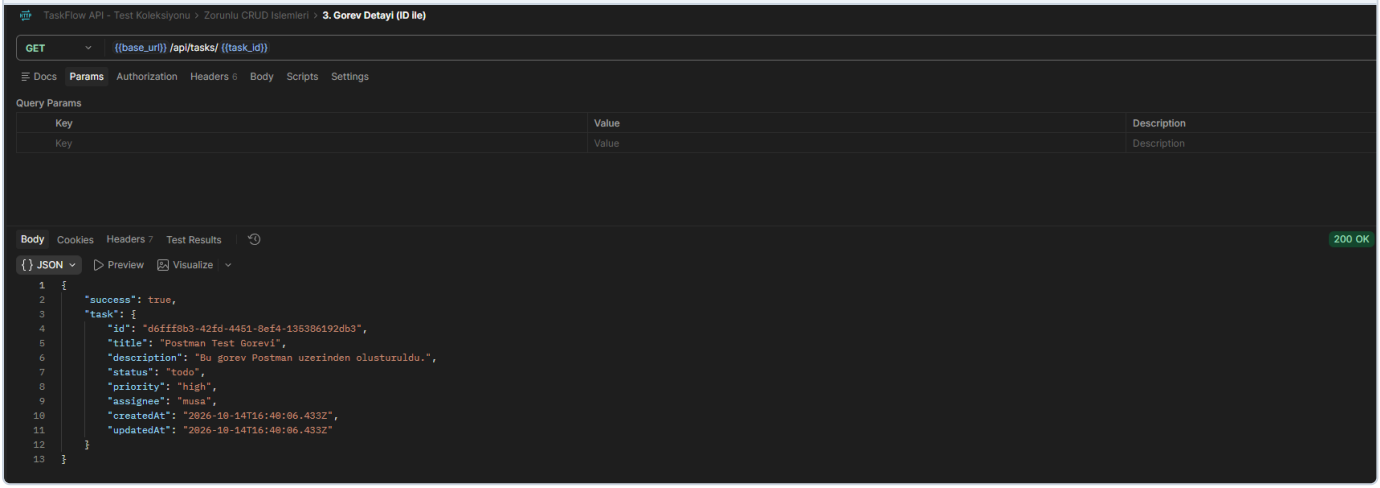
### 1. Tüm Görevlerin Listelenmesi (GET /api/tasks — 200 OK)



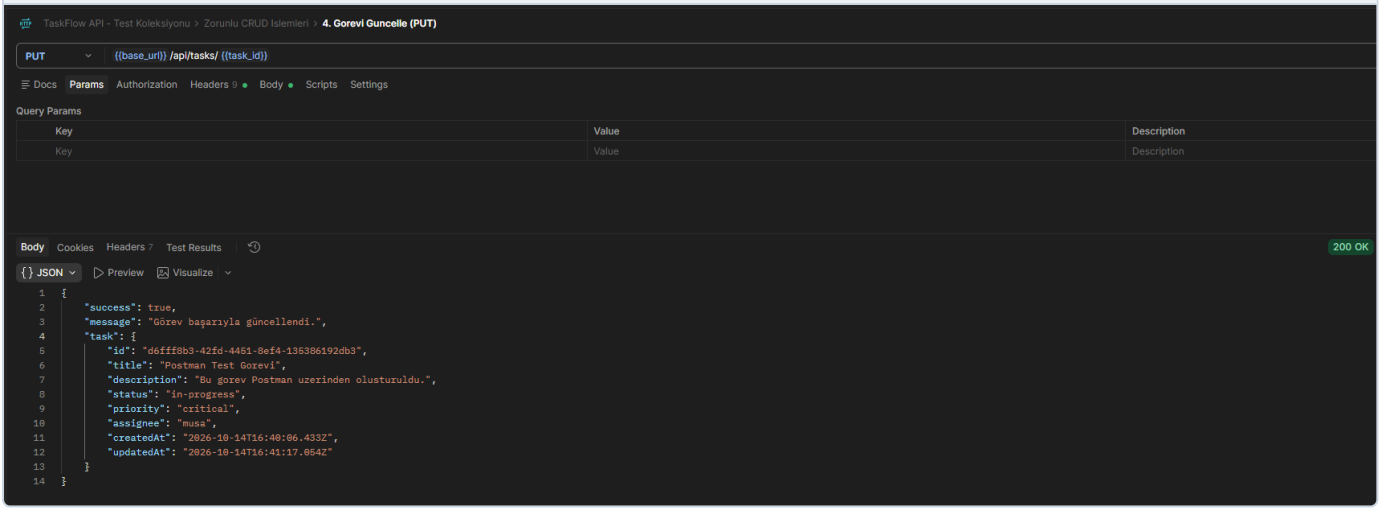

### 2. Yeni Görev Oluşturulması (POST /api/tasks — 201 Created)



## 3. Görev Detayının Getirilmesi (GET /api/tasks/:id — 200 OK)




## 4. Görevin Güncellenmesi (PUT /api/tasks/:id — 200 OK)




## 5. Görevin Silinmesi (DELETE /api/tasks/:id — 200 OK)

